

**LAPORAN BACK END
PROJECT BASED LEARNING
SISTEM BEBAS TANGGUNGAN TUGAS AKHIR**



Disusun oleh:

- | | |
|---|-------------------------|
| 1. Abdul Aziz (Project Manager) | : (244107023009) |
| 2. Achmad Diaz Hikmal Baihaqi (UI/UX) | : (2341720228) |
| 3. Adinda Mirza Devani (Database) | : (2341720046) |
| 4. Adnan Arju Maulana Pasha (FrontEnd) | : (2341720107) |
| 5. Muhammad Naufal Pratomo (BackEnd) | : (2341720075) |

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2024**

BAB I

PENDAHULUAN

1.1. Latar Belakang

Proses penyelesaian Tugas Akhir (TA) merupakan salah satu tahapan penting bagi mahasiswa. Namun, hambatan administratif seperti pengelolaan dokumen bebas tanggungan sering menjadi kendala. Dokumen-dokumen seperti tanda terima TA, tanda terima PKL, dan surat bebas kompensasi kerap kali dikelola secara manual, yang berpotensi menyebabkan kehilangan data, kesalahan verifikasi, dan keterlambatan proses. Kendala-kendala ini dapat mempengaruhi kelancaran mahasiswa dalam menyelesaikan tugas akhirnya.

Sebagai solusi, Sistem Bebas Tanggungan Tugas Akhir dikembangkan untuk mengotomatisasi proses administratif, meningkatkan efisiensi, dan memastikan akurasi data. Sistem ini dirancang dengan pendekatan Project-Based Learning (PBL), yang memungkinkan mahasiswa untuk terlibat aktif dalam perancangan dan implementasi sistem. Melalui PBL, mahasiswa dapat mengembangkan keterampilan praktis dan pemahaman yang mendalam dalam menyelesaikan masalah nyata terkait proses administrasi tugas akhir.

Fitur-fitur dalam sistem ini, seperti pencatatan log aktivitas, pengelolaan dokumen pendukung, dan pemanfaatan trigger untuk menjaga integritas data, memberikan transparansi dan kemudahan bagi pengguna. Dengan adanya Sistem Bebas Tanggungan Tugas Akhir, diharapkan proses administrasi dapat berjalan lebih efektif, mendukung mahasiswa dalam mencapai kelulusan tepat waktu, sekaligus memberikan pengalaman pembelajaran yang bermanfaat bagi pengembangan sistem.

BAB II

LANDASAN TEORI

2.1. Teknologi

2.1.1. Bahasa Pemrograman

Sistem informasi Bebas Tanggungan ini menggunakan Bahasa *PHP* digunakan untuk membangun sisi sistem, mencakup logika utama aplikasi dan interaksi dengan database dan JavaScript digunakan untuk mendukung fitur interaktif pada sisi user, terutama untuk validasi form atau interaksi dinamis lainnya.

2.1.2. Library

Sistem Informasi Bebas Tanggungan menggunakan beberapa library tambahan untuk menunjang proses develop system yaitu diantaranya :

- **jQuery:** Digunakan untuk manipulasi DOM dan mengelola permintaan AJAX.
- **Phpoffice:** Digunakan untuk memproses dokumen berbasis Excel atau Word pada sistem, memungkinkan admin untuk mengimpor atau mengekspor data dalam format yang mudah diakses.

Mpdf: Digunakan untuk menghasilkan file PDF dari dokumen yang dihasilkan sistem, seperti laporan verifikasi atau surat bebas tanggungan yang dapat diunduh oleh mahasiswa.

2.1.3. Tools

Dalam pembangunan sistem informasi bebas tanggungan menggunakan beberapa tools pendukung yaitu diantaranya:

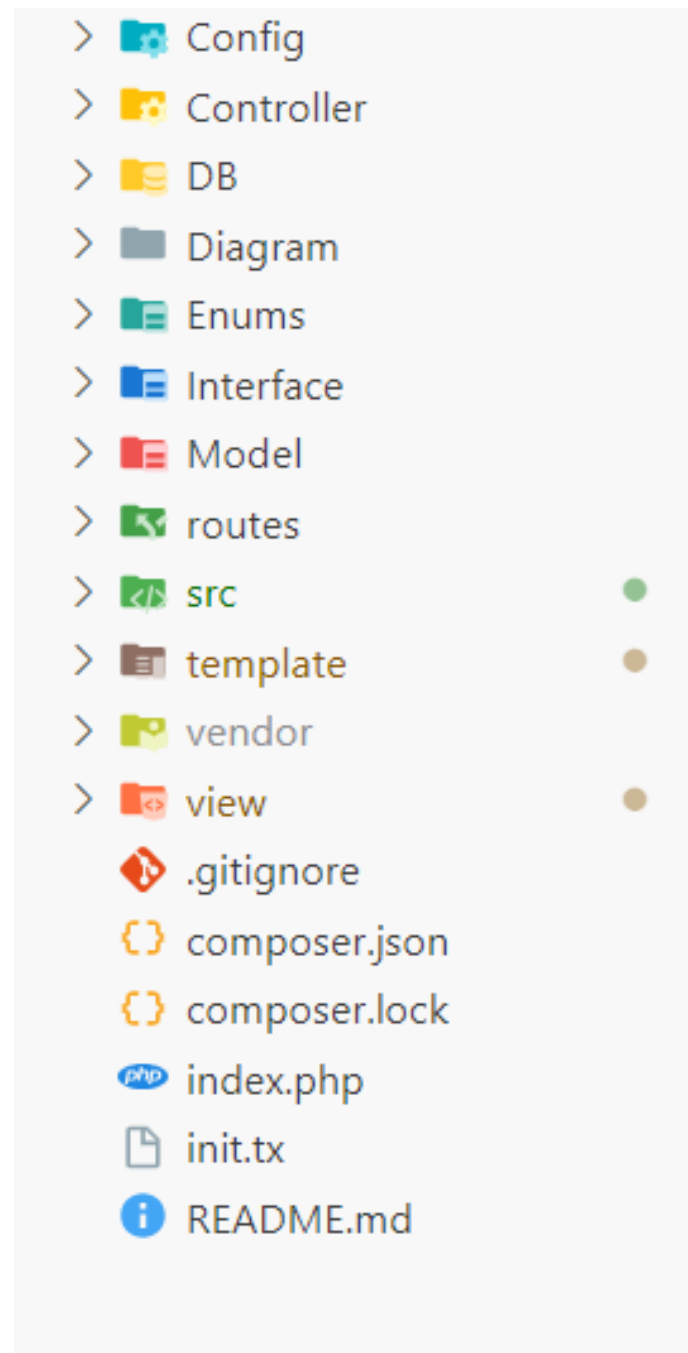
- **Git:** Untuk version control dan kolaborasi pengembangan.
- **Visual Studio Code:** Sebagai IDE utama untuk menulis dan mengelola kode backend.
- **SQL Server Management Studio:** Untuk mengelola dan menguji database sistem.
- **Google Chrome:** Digunakan sebagai browser utama untuk melakukan pengujian sistem, memastikan antarmuka pengguna berfungsi dengan baik di platform web modern.

BAB III

STRUKTUR DAN ARSITEKTUR

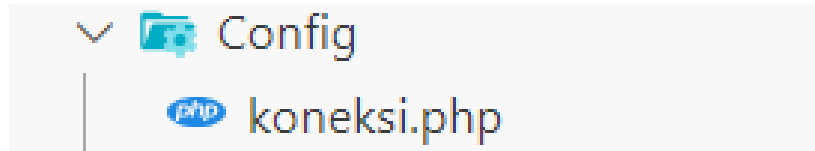
3.1. Struktur Folder

Struktur folder dan file pada backend dirancang untuk memisahkan responsibilities setiap komponen, sehingga memudahkan pengelolaan kode, skalabilitas, dan pemeliharaan. Berikut adalah struktur folder umum yang digunakan:



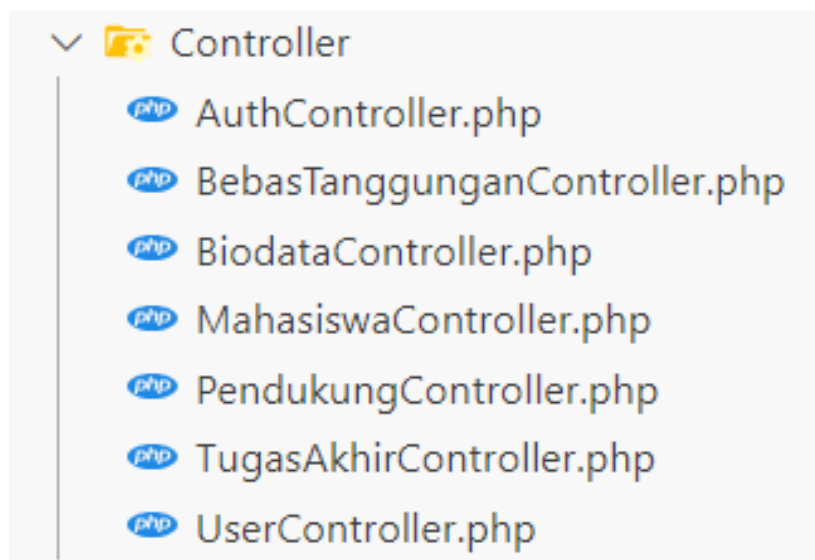
3.1.1. Config

Folder config digunakan untuk beberapa file yang menangani masalah config yang diperlukan oleh system. Config berisi sebuah class Koneksi untuk menangani masalah koneksi dengan database.



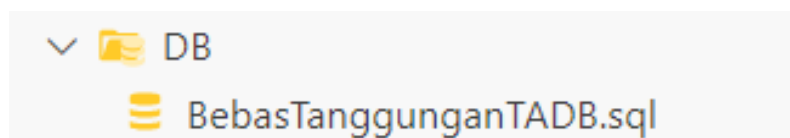
3.1.2. Controller

Folder Controller berisi file controller yang digunakan dalam system, controller sendiri berfungsi untuk menjembatani antara view dan model dalam design pattern MVC dan berisi logika bisnis yang diperlukan system.



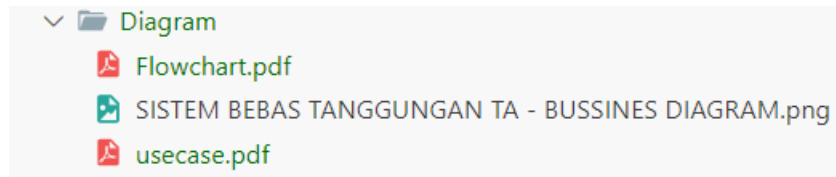
3.1.3. DB

Folder db berisi file SQL yang merupakan database yang akan digunakan dalam system hal ini memudahkan untuk tim dalam proses pembuatan system karena semua database yang dimiliki tim akan sama dan tim tidak perlu lagi untuk mengirim file sql secara manual.



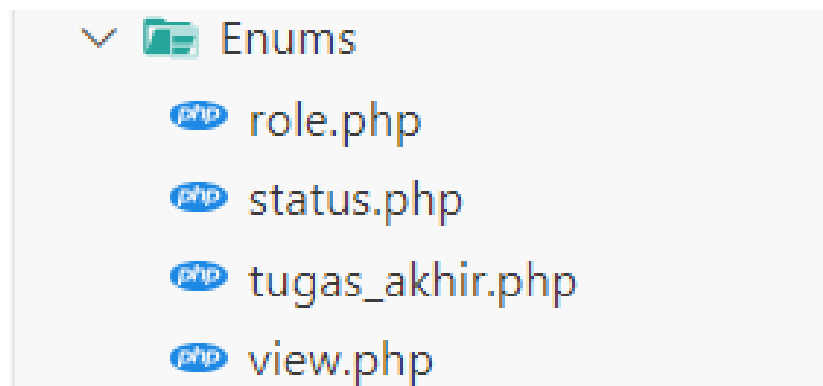
3.1.4. Diagram

Semua diagram yang akan digunakan dalam pembangunan system akan diletakkan didalam folder diagram ini. Hal ini bertujuan untuk menyamakan perspektif semua anggota tim dalam membangun system.



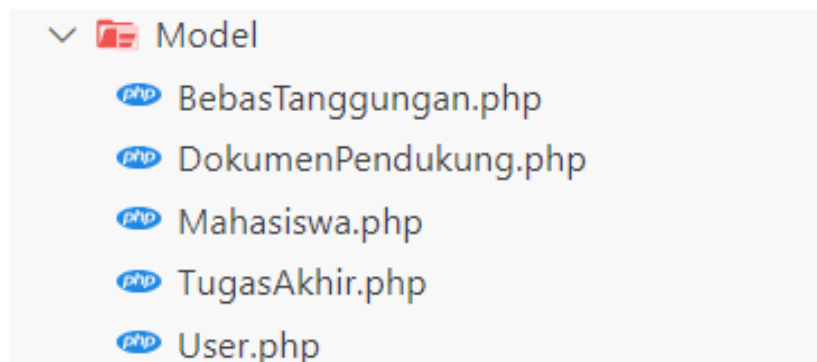
3.1.5. Enums

folder Enums berisi file PHP yang digunakan untuk mendefinisikan enumerasi (*Enums*). Di PHP, *Enums* berfungsi untuk mendeklarasikan kumpulan nilai tetap yang digunakan untuk merepresentasikan status, peran, atau kategori dalam aplikasi. Hal ini membantu meningkatkan keterbacaan kode dan mengurangi potensi kesalahan karena penggunaan nilai *hardcoded*.



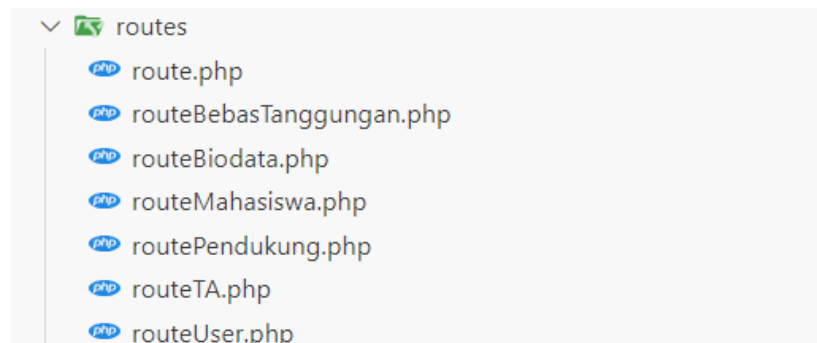
3.1.6. Model

Folder model berisi file PHP yang digunakan untuk menangani data dan logika bisnis system, termasuk berinteraksi dengan database dan melakukan operasi terkait data.



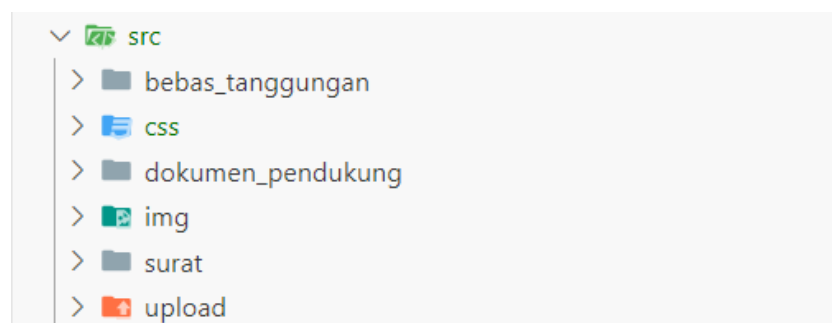
3.1.7. Routes

Folder routes berisi file PHP yang digunakan untuk mengatasi route yang diakses oleh user dan menangani request dari user dan diproses oleh system, route juga berfungsi untuk memisahkan logika URL dan Controller.



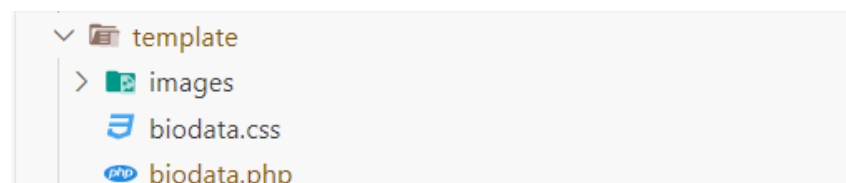
3.1.8. Src

Folder src berisi beberapa file yang dibutuhkan terutama yang terkait dengan media seperti image, pdf, dan lain-lain. Selain itu folder ini juga berisi file-file yang telah diupload user dalam system seperti foto profil, dan dokumen-dokumen dan berisi file css yang dibutuhkan system.



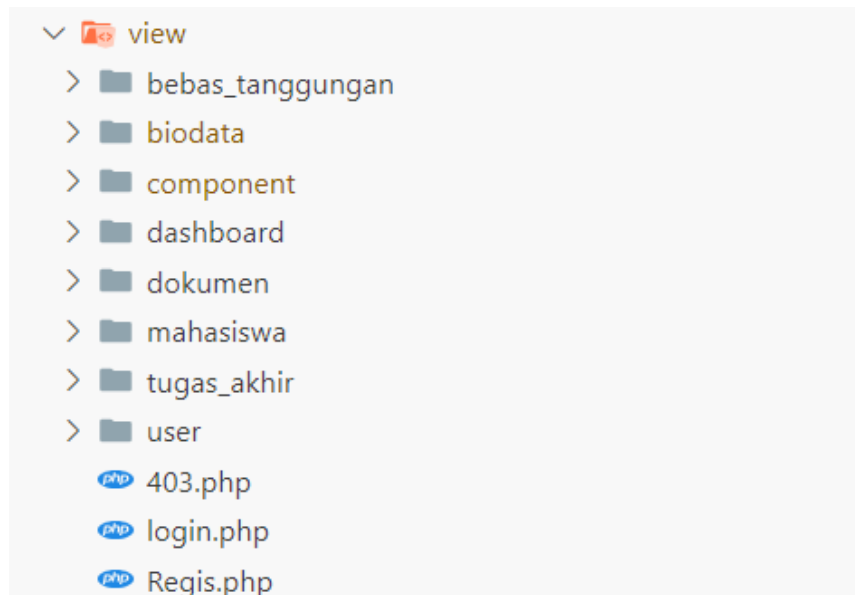
3.1.9. Template

Folder ini berisi tampilan yang sementara yang dibuat oleh tim frontend sebelum dilakukan proses slicing atau templating pada system, hal ini berguna agar masing-masing tim tetap bisa berkolaborasi dalam satu waktu tanpa harus menunggu hasil dari tim yang lain.



3.1.10. View

Folder ini berisi semua tampilan-tampilan yang akan digunakan dalam system dalam menampilkan view menggunakan mekanisme templating.



3.2. Arsitektur Sistem

Model-View-Controller (MVC) adalah salah satu pendekatan arsitektur backend yang populer karena memisahkan logika aplikasi menjadi tiga lapisan utama:

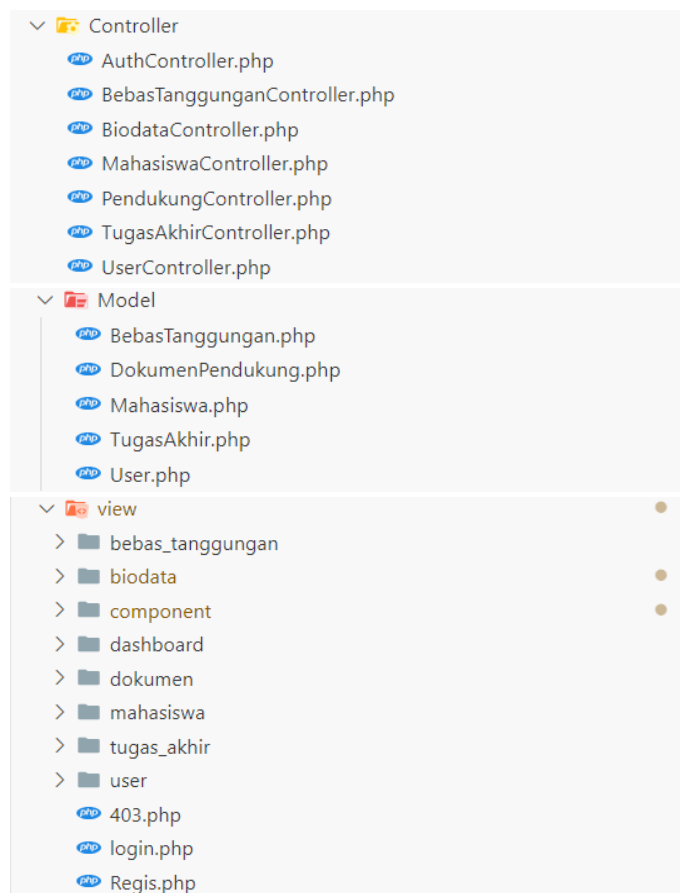
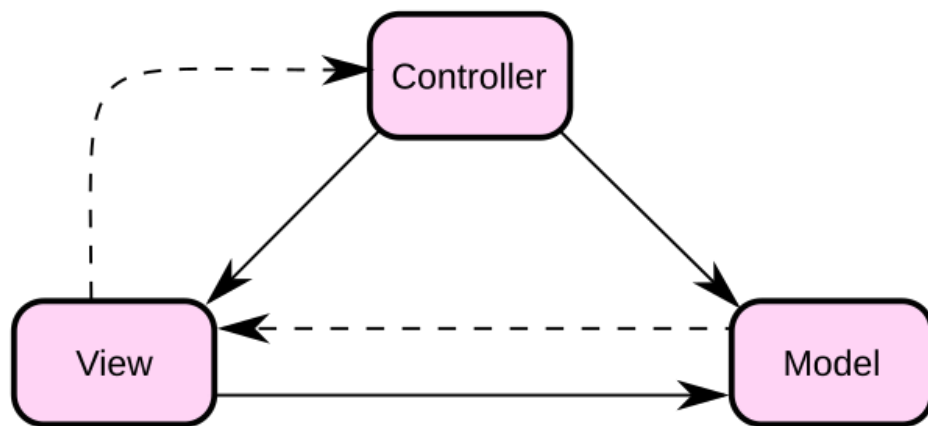
1. **Model**
 - Berhubungan dengan data, struktur database, dan interaksi dengan basis data.
2. **View**
 - Biasanya tidak ada pada backend API murni, karena frontend yang menangani presentasi.
 - Untuk backend, "View" dapat diartikan sebagai respon API (JSON).
3. **Controller**
 - Mengelola logika utama dan bertindak sebagai penghubung antara *Model* dan *View*.
 - Contoh: Mengambil data dari *Model* dan mengirimkan respon JSON atau array ke pengguna.

Alur Kerja

1. **Client Request:** Frontend atau client lain mengirimkan request.
2. **Router:** Mengarahkan permintaan ke *Controller* yang sesuai berdasarkan endpoint.
3. **Controller:** Mengambil data atau melakukan operasi yang diperlukan dengan bantuan *Model*.
4. **Model:** Berkomunikasi dengan database untuk membaca/menulis data.
5. **Response:** Controller memformat data menjadi respon dan mengirimkannya kembali ke client.

Keuntungan Pendekatan MVC

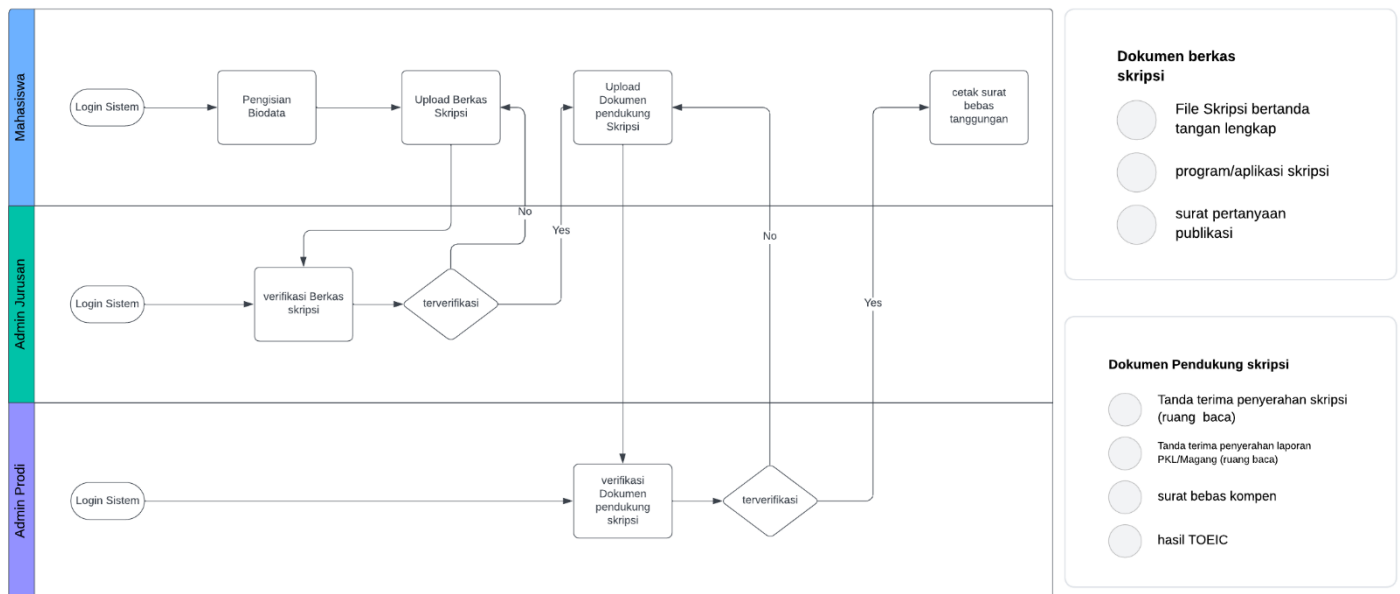
- **Organisasi Kode:** Memisahkan tugas, membuat kode lebih rapi dan mudah dibaca.
- **Skalabilitas:** Mudah menambahkan fitur baru tanpa merusak struktur yang ada.
- **Kolaborasi:** Setiap anggota tim dapat bekerja pada bagian tertentu (misalnya, developer frontend fokus pada View, sementara backend mengerjakan Model dan Controller).



3.3. Proses Bisnis

Proses bisnis system bebas tanggungan ini mencakup system yang digunakan mahasiswa untuk mendapatkan surat bebas tanggungan dengan syarat mengupload dokumen-dokumen yang dibutuhkan kemudian dilakukannya proses verifikasi dari admin dan akan menghasilkan output surat bebas tanggungan.

**BUSSINESS DIAGRAM
SISTEM BEBAS TANGGUNGAN TA**



3.3. Use Case Diagram

Dalam system informasi bebas tanggungan ini memiliki beberapa fitur yang akan dibagi menjadi 4 role yaitu super admin, mahasiswa, admin jurusan, dan admin prodi dan setiap role memiliki fitur tersendiri.

1. Role Super Admin

- Management data user, fitur ini digunakan super admin untuk mengatur user terutama terkait dengan akun system dan role akun.
- Management data mahasiswa, fitur ini digunakan super admin untuk mengatur data mahasiswa yang berupa biodata mahasiswa.
- Management data Dokumen Tugas Akhir, fitur ini digunakan super admin untuk mengatur Dokumen Tugas Akhir mulai dari upload, edit, hapus, dan verifikasi data dokumen tugas akhir.
- Management data Dokumen Pendukung, fitur ini digunakan super admin untuk mengatur Dokumen pendukung mulai dari upload, edit, hapus, dan verifikasi data dokumen tugas akhir.
- Management Bebas Tanggungan, fitur ini digunakan super admin untuk mengatur data bebas tanggungan yang mencakup

pembuatan request surat, pembuatan surat bebas tanggungan, dan verifikasi surat bebas tanggungan.

- Management Arsip, fitur ini digunakan super admin untuk melihat dokumen-dokumen mahasiswa yang terverifikasi.

2. Role Mahasiswa

- Management Biodata, fitur ini digunakan mahasiswa untuk mengupload atau mengupdate data biodata mahasiswa itu sendiri.
- Management Dokumen Tugas Akhir, fitur ini digunakan mahasiswa untuk mengupload dan meninjau proses verifikasi dokumen tugas akhir sebagai syarat bebas tanggungan.
- Management Dokumen Pendukung, fitur ini digunakan mahasiswa untuk mengupload dan meninjau proses verifikasi dokumen pendukung sebagai syarat bebas tanggungan.
- Management Bebas Tanggungan, fitur ini digunakan mahasiswa untuk melakukan request surat bebas tanggungan dan mendownload surat bebas tanggungan, bebas tanggungan akan tercetak jika semua dokumen telah terverifikasi.

3. Role Admin Jurusan

- Management data mahasiswa, fitur ini digunakan admin jurusan untuk mengatur data mahasiswa yang berupa biodata mahasiswa.
- Management data Dokumen Tugas Akhir, fitur ini digunakan admin jurusan untuk mengatur Dokumen Tugas Akhir mulai dari upload, edit, hapus, dan verifikasi data dokumen tugas akhir.
- Management Bebas Tanggungan, fitur ini digunakan admin jurusan untuk mengatur data bebas tanggungan yang mencakup pembuatan request surat, pembuatan surat bebas tanggungan, dan verifikasi surat bebas tanggungan.
- Management Arsip, fitur ini digunakan admin jurusan untuk melihat dokumen-dokumen mahasiswa yang terverifikasi.

4. Role Admin Prodi

- Management data mahasiswa, fitur ini digunakan admin prodi untuk mengatur data mahasiswa yang berupa biodata mahasiswa.
- Management data Dokumen Pendukung, fitur ini digunakan admin prodi untuk mengatur Dokumen pendukung mulai dari upload, edit, hapus, dan verifikasi data dokumen tugas akhir.
- Management Arsip, fitur ini digunakan admin prodi untuk melihat dokumen-dokumen mahasiswa yang terverifikasi.

Secara umum, penyusunan use case diagram melalui beberapa tahapan, seperti:

1) Definisi Aktor

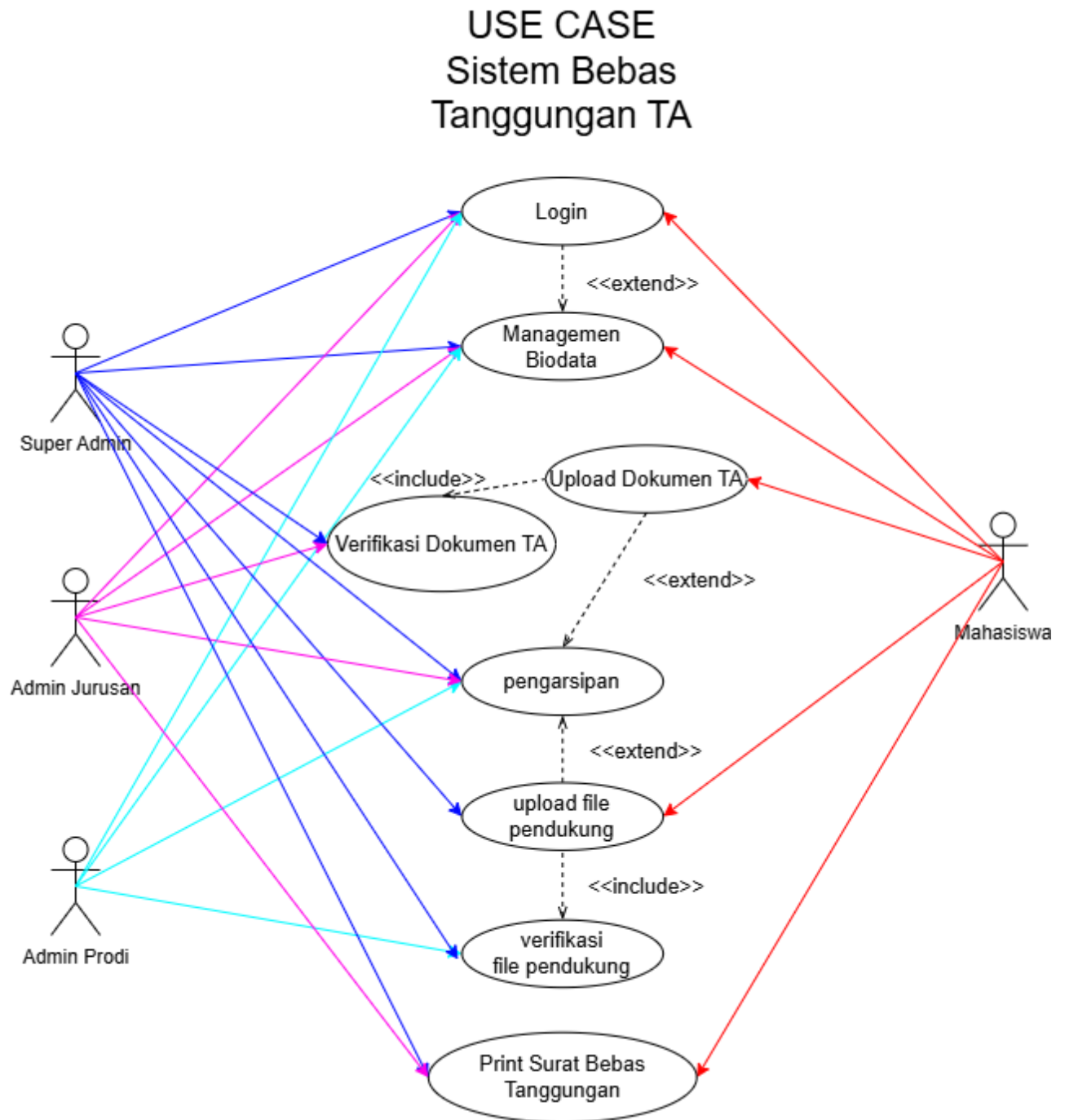
No	Aktor	Deskripsi
1	Super Admin	Super Admin adalah orang yang bertugas untuk memmanagement semua data master mulai dari data user, mahasiswa, bebas tanggungan. Selain itu super admin juga bisa melakukan verifikasi terkait dokumen yang diupload mahasiswa.
2	Mahasiswa	Mahasiswa adalah orang yang bertugas untuk mengupload semua dokumen yang berkaitan dengan bebas tanggungan dan merequest surat bebas tanggungan.
3	Admin Jurusan	Admin Jurusan adalah orang yang bertugas untuk melakukan verifikasi dokumen tugas akhir mahasiswa yang telah diupload, selain itu admin jurusan juga bertugas untuk mencetak atau membuat surat bebas tanggungan yang direquest oleh mahasiswa.
4	Admin Prodi	Admin prodi adalah orang yang bertugas untuk melakukan verifikasi dokumen pendukung yang telah diupload oleh mahasiswa.

2) Definisi Use Case

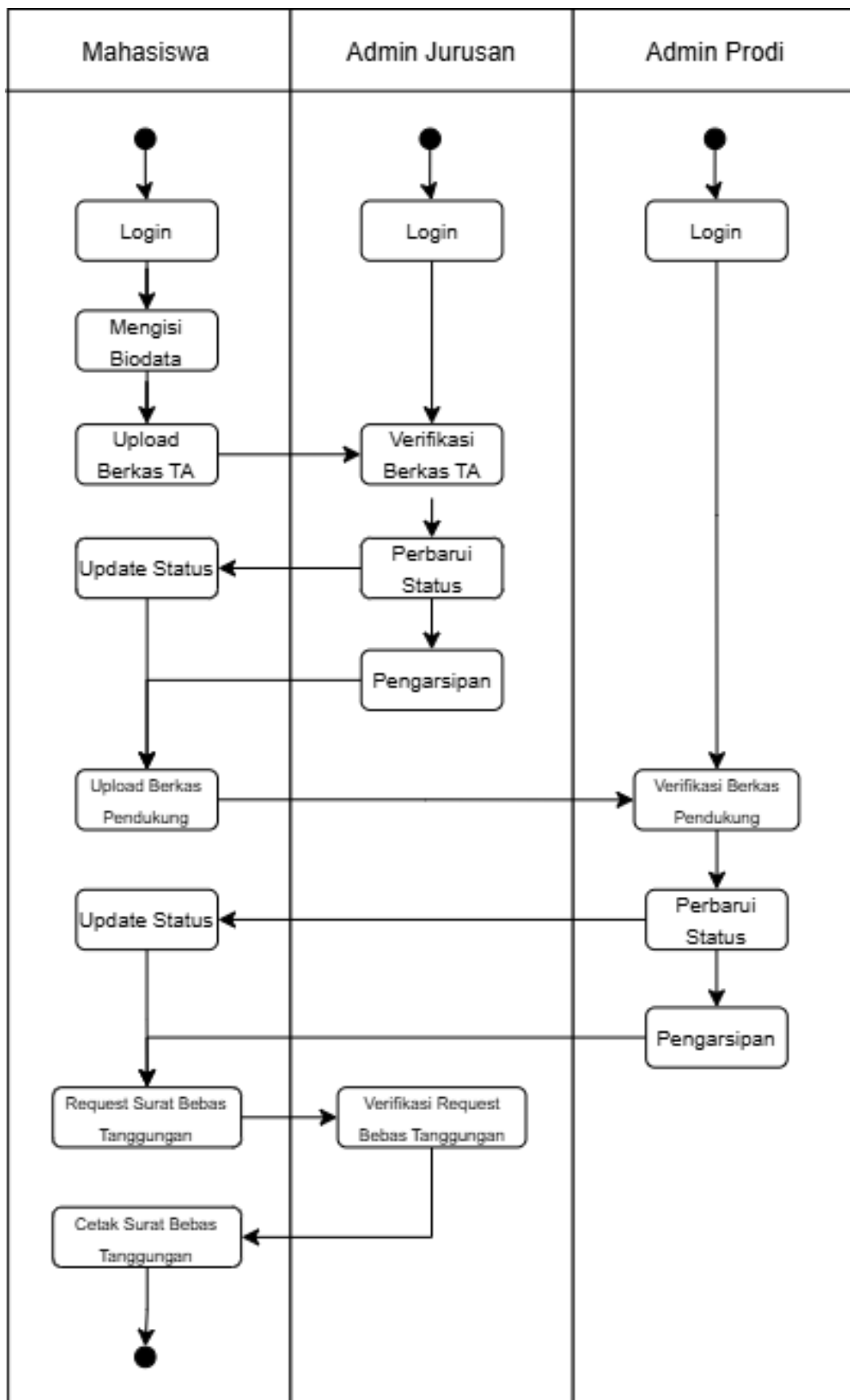
No	Use Case	Deskripsi
1	Login	Proses awal masuk system sebelum menggunakan fitur-fitur yang ada.
2	Management Biodata	Proses management biodata atau mahasiswa yang mencakup proses tambah, edit, dan hapus data mahasiswa.
3	Upload Dokumen Tugas Akhir	Proses upload Dokumen Tugas Akhir yang bisa dilakukan oleh 3 role yaitu mahasiswa, super admin, dan admin jurusan. Untuk role super admin dan admin jurusan bisa melakukan hapus data.
4	Verifikasi Dokumen Tugas Akhir	Proses ini merupakan proses verifikasi Dokumen Tugas Akhir yang mencakup menambahkan note atau catatan pada setiap dokumen yang perlu

		adanya upload ulang atau perbaikan. Proses ini bisa dilakukan oleh role super admin dan admin jurusan.
5	Upload Dokumen Pendukung	Proses upload Dokumen Pendukung yang bisa dilakukan oleh 3 role yaitu mahasiswa, super admin, dan admin prodi. Untuk role super admin dan admin prodi bisa melakukan hapus data.
6	Verifikasi Dokumen Tugas Akhir	Proses ini merupakan proses verifikasi Dokumen Pendukung yang mencakup menambahkan note atau catatan pada setiap dokumen yang perlu adanya upload ulang atau perbaikan. Proses ini bisa dilakukan oleh role super admin dan admin prodi.
7	Pengarsipan	Proses ini merupakan proses untuk melihat dokumen-dokumen mahasiswa yang telah terverifikasi. Proses ini bisa dilakukan oleh role admin, baik itu super admin, admin jurusan, ataupun admin prodi.

3) Use Case Diagram



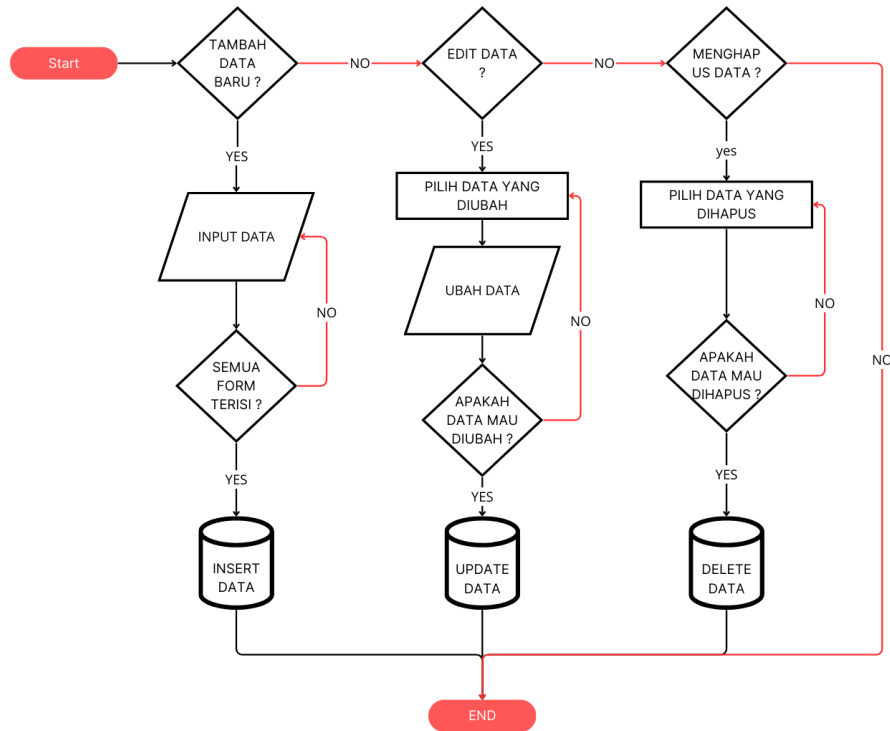
3.4. Flow Chart



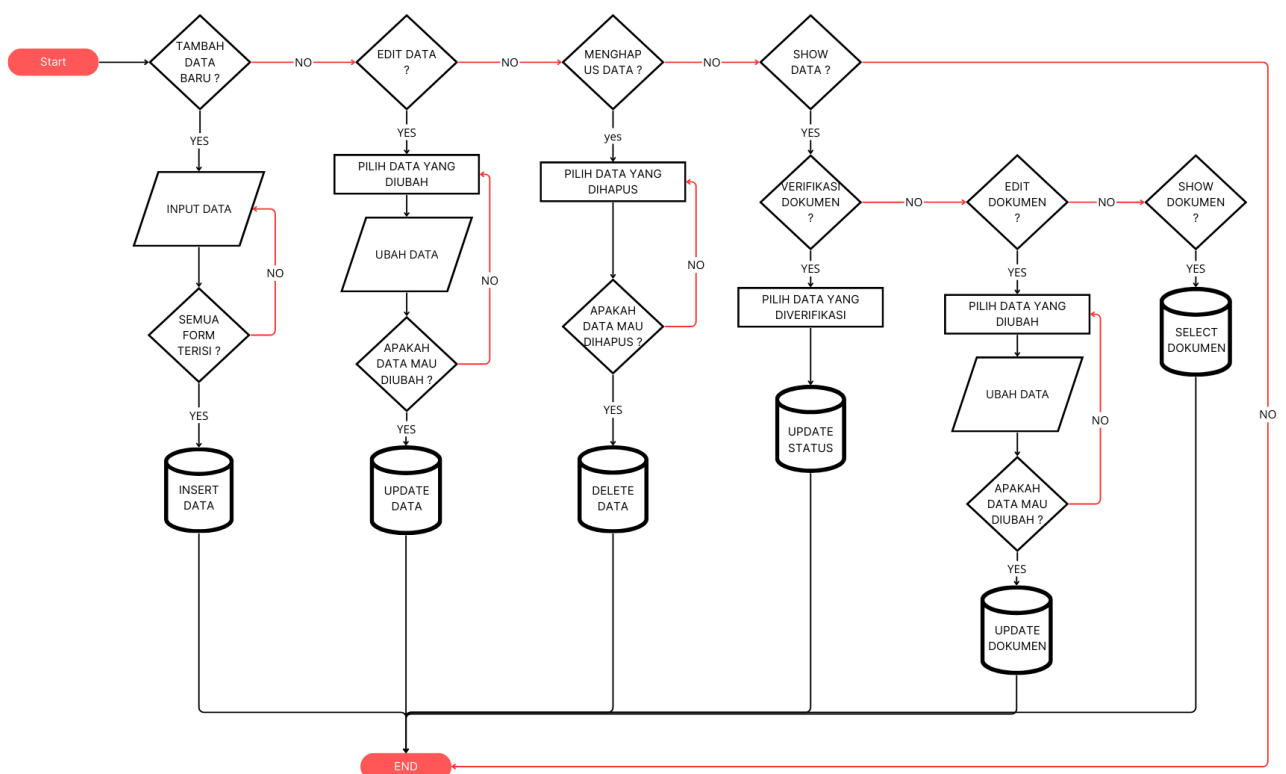
3.5. Diagram Alir

Semua fitur yang ada didalam aplikasi ini memiliki proses yang berbeda. Berikut ini adalah diagram alir dari masing-masing fitur yang ada :

1) Management User dan Mahasiswa



2) Management Dokumen Tugas Akhir



BAB IV

IMPLEMENTASI BACKEND

4.1. Fitur Otentikasi dan Otorisasi

Fitur ini menangani proses login pengguna dan memastikan setiap permintaan sesuai dengan hak akses pengguna.

Letak Source Code:

- Controller: controller/AuthController.php

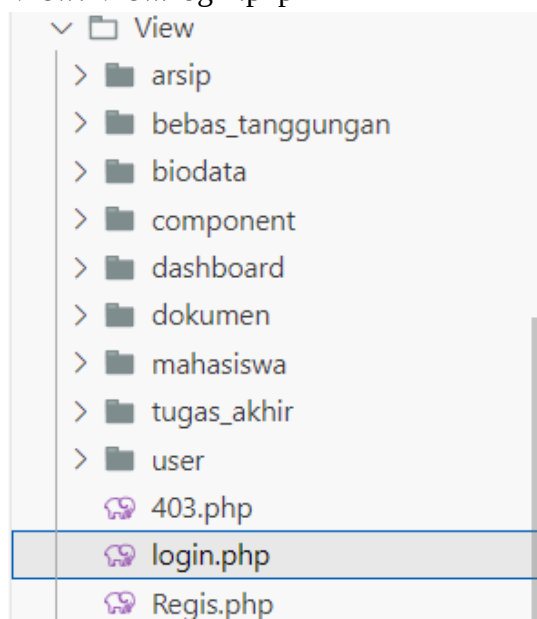
Berisi fungsi proses_login() untuk memverifikasi kredensial pengguna dan menyimpan data pengguna ke dalam sesi.

```
public function proses_login(): void {
    $nim = $_POST['nim'];
    $password = $_POST['password'];
    $validasi = $this->user->validasi_nim(nim: $nim, password: $password);
    if($validasi == true) {
        header(header: 'location:../view/dashboard/index.php');
    } else {
        header(header: 'location:../view/login.php');
    }
}
```

- Model: Model/User.php

```
public function validasi_nim($nim, $password): bool {
    $query = "select * from dbo.Users where username = '$nim' and password = '$password'";
    $data = $this->koneksi->KoneksiDB()->query($query);
    $result = $data->fetch();
    if(empty($result) == true) {
        $_SESSION['error'] = "Nim atau Password salah";
        return false;
    } else {
        $_SESSION['user'] = $result;
        return true;
    }
}
```

- View: View/login.php



- Route: Routes/route.php

```
if ($page == 'login') {
    $auth->login();
} elseif ($page == 'proses_login') {
    $auth->proses_login();
} elseif ($page == 'logout') {
    $auth->logout();
}
```

4.2. Fitur Manajemen Data User

Fitur ini menangani proses Management data User yang mencakup tambah, edit, dan hapus user.

Letak Source Code:

- Controller: controller/UserController.php

Berisi beberapa fungsi untuk menunjang proses management data user seperti fungsi addUser(), getAll(), dan hapus().

```
class UserController implements UserControllerInterface {
    7 references
    private $user;
    1 reference
    private $mahasiswa;

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->user = new User();
        $this->mahasiswa = new Mahasiswa();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function addUser(): void{
        $username = $_POST['username'];
        $email = $_POST['email'];
        $password = $_POST['password'];
        $role = $_POST['role'];
        $data = $this->user->addUser(username: $username, email: $email, password: $password, role: $role);
        if($data == true) {
            $_SESSION['sukses'] = "Data Berhasil Ditambahkan";
            header(header: 'location: ../routes/route.php?page=user&sub=manageuser');
        } else {
            $_SESSION['error'] = "Data Gagal Ditambahkan";
            header(header: 'location: ../routes/route.php?page=user&sub=manageuser');
        }
    }
}
```

- Model: Model/User.php

Berisi sebuah query sql yang menghubungkan antara sistem dan database untuk keperluan fitur management user.

```

class user {
    8 references
    private $koneksi;

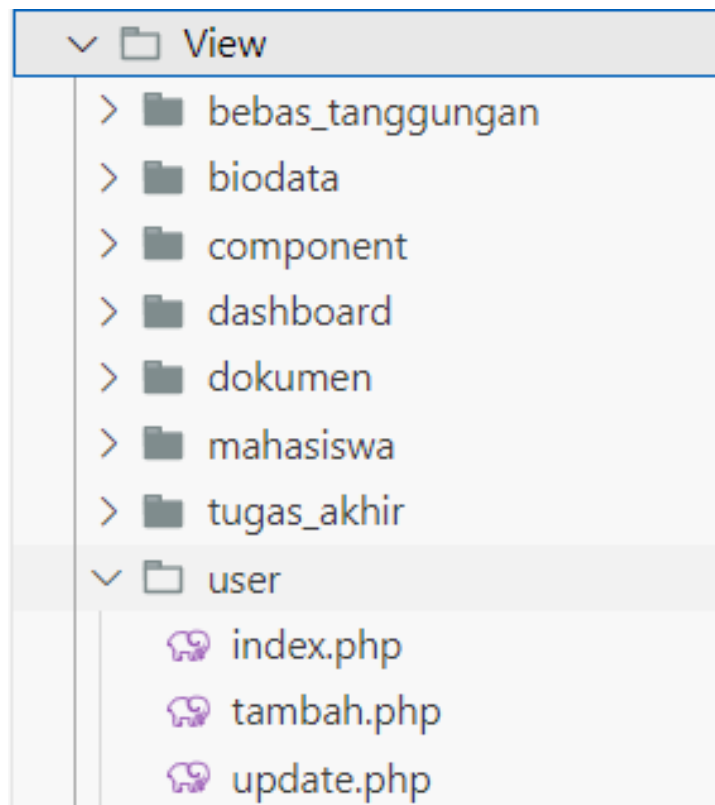
    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct(){
        $this->koneksi = new Koneksi();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function validasi_nim($nim, $password): bool {
        $query = "select * from dbo.Users where username = '$nim' and password = '$password'";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetch();
        if(empty($result) == true) {
            $_SESSION['error'] = "Nim atau Password salah";
            return false;
        } else {
            $_SESSION['user'] = $result;
            return true;
        }
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function addUser($username, $email, $password, $role): bool {
        try {
            $query = "INSERT INTO Users (username, email, password, role)
            VALUES ('$username', '$email', '$password', '$role')";
            $data = $this->koneksi->KoneksiDB()->query($query);
            return true;
        } catch(PDOException $e) {
            return false;
        }
    }
}

```

- View: View/user



- Route: Routes/route.php dan Routes/routeUser.php

```

if ($page == 'user') {
    $user->route();
}

class routeUser {
    7 references
    private $user;
    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct(){
        $this->user = new UserController();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function route(): void {
        $sub = isset($_GET['sub']) ? $_GET['sub'] : $_GET['sub'] = 'manageuser';
        $id = isset($_GET['id']) ? $_GET['id'] : "";
        if($sub == 'manageuser') {
            header(header: 'location:../view/' . view::USER->value . '/index.php');
        } elseif ($sub == 'tambahuser') {
            $this->user->addUser();
        } elseif ($sub == 'getAll') {
            $this->user->getAll();
        } elseif($sub == 'edit') {
            $this->user->edit(id: $id);
        } elseif($sub == 'getOne') {
            $this->user->getOne(id: $id);
        } elseif($sub == 'update') {
            $this->user->update(id: $id);
        } elseif($sub == 'hapus') {
            $this->user->delete(id: $id);
        }
    }
}

```

4.3. Fitur Management data mahasiswa

Fitur ini memungkinkan user bisa mengakses dan memodifikasi data mahasiswa yang mencakup tambah, edit, dan hapus data mahasiswa.

Letak Source Code:

- Controller: controller/MahasiswaController.php
Berisi beberapa fungsi untuk menunjang proses management data user seperti fungsi addUser(), getAll(), dan hapus().

```

class MahasiswaController
{
    9 references
    private $mahasiswa;
    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->mahasiswa = new Mahasiswa();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): void
    {
        $data = $this->mahasiswa->getAll();
        if ($data) {
            header(header: 'Content-Type: application/json');
            echo json_encode(value: $data);
        } else {
            echo false;
        }
    }
}

```

- Model: Model/Mahasiswa.php
Berisi sebuah query sql yang menghubungkan antara sistem dan database untuk keperluan fitur management mahasiswa.

```

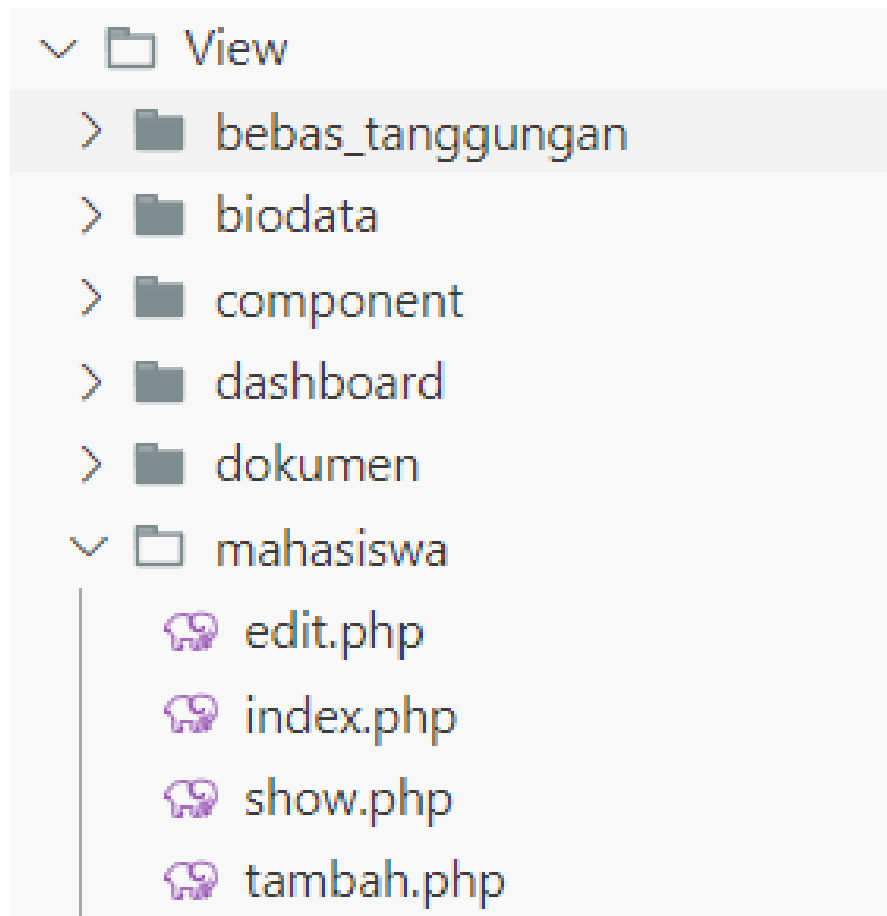
class Mahasiswa
{
    9 references
    private $koneksi;
    6 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->koneksi = new koneksi();
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getOne($id): mixed
    {
        $query = "SELECT * FROM Mahasiswa WHERE mahasiswa_id = $id";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetch();
        return $result;
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getUser($id): mixed
    {
        $query = "SELECT *
        FROM Mahasiswa as mhs
        inner join Users as usr on usr.user_id = mhs.user_id
        where mhs.mahasiswa_id = $id";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetch();
        return $result;
    }
}

```

- View: View/Mahasiswa



- Route: Routes/route.php dan Routes/routeMahasiswa.php

```
elseif ($page == 'mahasiswa') {  
    $mahasiswa->route();  
}
```

```

class routeMahasiswa {
    9 references
    private $mahasiswa;
    2 references
    private $user;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->mahasiswa = new MahasiswaController();
        $this->user = new UserController();
    }

    4 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function route(): void {
        $sub = isset($_GET['sub']) ? $_GET['sub'] : $_GET['sub'] = 'managemahasiswa';
        $id = isset($_GET['id']) ? $_GET['id'] : "";
        if($sub == "managemahasiswa") {
            header(header: 'location:../view/' . view::MAHASISWA->value . '/index.php');
        } elseif($sub == "getAll") {
            $this->mahasiswa->getAll();
        } elseif($sub == "show") {
            $this->mahasiswa->show(id: $id);
        } elseif($sub == "getUser") {
            $this->user->getEmpty();
        } elseif($sub == "addMahasiswa") {
            $this->mahasiswa->addMahasiswa();
        } elseif($sub == "edit") {
            $this->mahasiswa->edit(id: $id);
        } elseif($sub == 'getWithUser') {
            $this->mahasiswa->getOne(id: $id);
        } elseif($sub == 'getByUser') {
            $this->mahasiswa->getByUser(id: $id);
        } elseif($sub == 'updateMahasiswa') {
            $this->mahasiswa->update(id: $id);
        } elseif($sub == 'hapus') {
            $this->mahasiswa->hapus(id: $id);
        }
    }
}

```

4.4. Fitur Management data Dokumen Tugas Akhir

Fitur ini memungkinkan user bisa mengakses dan memodifikasi data Dokumen Tugas Akhir yang mencakup tambah, edit, hapus, dan verifikasi dokumen tugas akhir.

Letak Source Code:

- Controller: controller/TugasAkhirController.php
Berisi beberapa fungsi untuk menunjang proses management Dokumen Tugas akhir seperti fungsi addUser(), getAll(), dan hapus().

```

class TugasAkhirController {
    26 references
    private $tugas_akhir;
    2 references
    private $mahasiswa;
    1 reference
    private $user;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->tugas_akhir = new TugasAkhir();
        $this->mahasiswa = new Mahasiswa();
        $this->user = $_SESSION['user']['user_id'];
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function index(): void {
        header(header: 'location:../view/' . View::TUGASAKHIR->value . '/index.php');
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): void {
        $data = $this->tugas_akhir->getAll();
        if($data) {
            header(header: 'Content-Type: application/json');
            echo json_encode(value: $data);
        } else {
            echo false;
        }
    }
}

```

- Model: Model/TugasAkhir.php
Berisi sebuah query sql yang menghubungkan antara sistem dan database untuk keperluan fitur management Dokumen Tugas Akhir.

```

class TugasAkhir
{
    27 references
    private $koneksi;

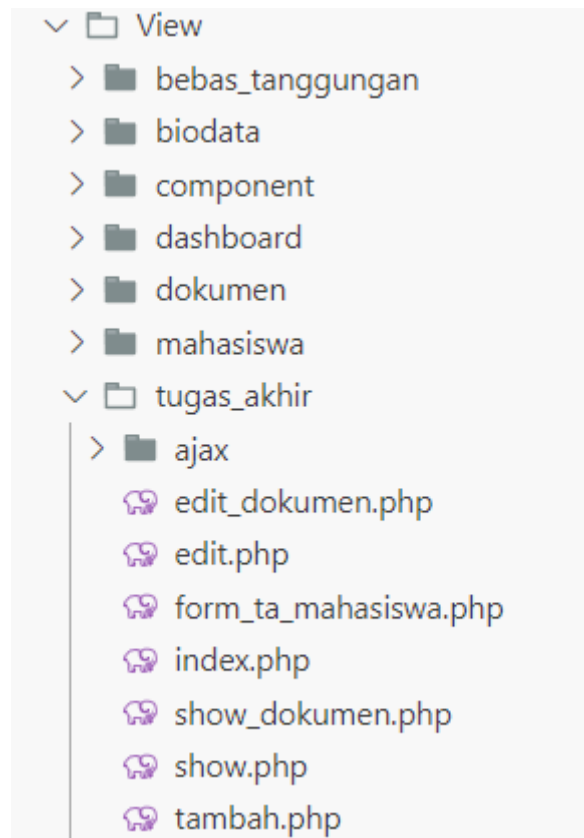
    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->koneksi = new koneksi();
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): array
    {
        $query = "select * from Tugas_akhir as ta inner join Mahasiswa as mhs
            on mhs.mahasiswa_id = ta.mahasiswa_id order by ta.tugas_akhir_id desc";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetchAll();
        return $result;
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getOne($id): mixed {
        $query = "SELECT * from Tugas_akhir as ta inner join Mahasiswa as mhs
            on mhs.mahasiswa_id = ta.mahasiswa_id WHERE tugas_akhir_id = $id";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetch();
        return $result;
    }
}

```


- View: View/tugas_akhir



- Route: Routes/route.php dan Routes/routeTA.php

```
elseif ($page == 'tugasakhir') {  
    $tugas_akhir->route();  
}
```

```

class routeTA {
    21 references
    private $tugas_akhir;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->tugas_akhir = new TugasAkhirController();
    }

    3 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function route(): void {
        // role super admin
        $sub = isset($_GET['sub']) ? $_GET['sub'] : $_GET['sub'] = 'manageTA';
        $id = isset($_GET['id']) ? $_GET['id'] : "";
        if($sub == 'manageTA') {
            $this->tugas_akhir->index();
        } elseif($sub == 'getAll') {
            $this->tugas_akhir->getAll();
        } elseif($sub == 'getOne') {
            $this->tugas_akhir->getOne(id: $id);
        } elseif($sub == 'edit') {

```

4.5. Fitur Management data Dokumen Pendukung

Fitur ini memungkinkan user bisa mengakses dan memodifikasi data Dokumen Pendukung yang mencakup tambah, edit, hapus, dan verifikasi dokumen Pendukung.

Letak Source Code:

- Controller: controller/PendukungController.php
Berisi beberapa fungsi untuk menunjang proses management Dokumen Pendukung seperti fungsi addUser(), getAll(), dan hapus().

```

class PendukungController {
    19 references
    private $DokumenPendukung;
    2 references
    private $mahasiswa;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->DokumenPendukung = new DokumenPendukung();
        $this->mahasiswa = new Mahasiswa();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): void {
        $data = $this->DokumenPendukung->getAll();
        if($data) {
            header(header: 'Content-Type: application/json');
            echo json_encode(value: $data);
        } else {
            echo false;
        }
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function add(): void {
        $file = [
            'tanda_terima_ta' => $_FILES['tanda_terima_ta'],
            'tanda_terima_pk1' => $_FILES['tanda_terima_pk1'],
            'bebas_kompen' => $_FILES['bebas_kompen']
        ];
        $tugas_akhir_data = $_POST['tugas_akhir'];
        list($tugas_akhir, $nim) = explode(separator: ':', string: $tugas_akhir_data);
        $targetDir = '../src/dokumen_pendukung/' . $nim . "/";
    }

```

- Model: Model/DokumenPendukung.php
Berisi sebuah query sql yang menghubungkan antara sistem dan database untuk keperluan fitur management Dokumen Pendukung.

```

class DokumenPendukung
{
    18 references
    private $koneksi;

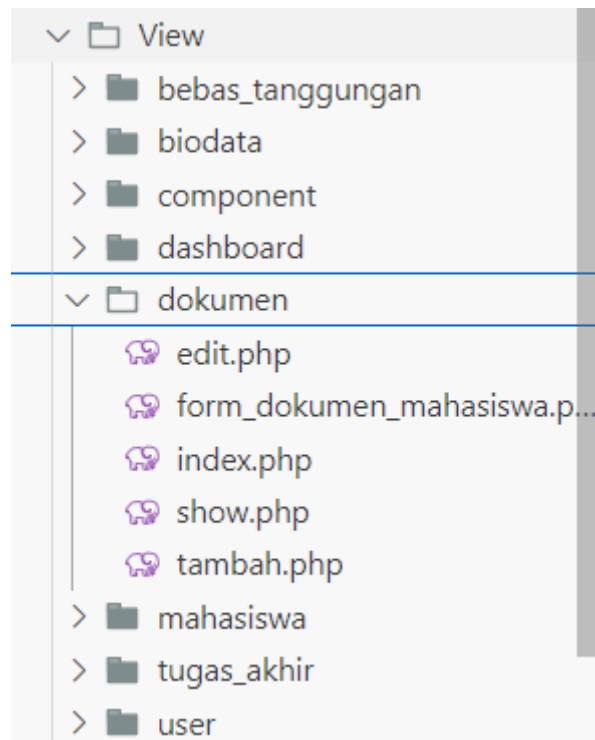
    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->koneksi = new Koneksi();
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): array
    {
        $query = "SELECT * FROM Dokumen_pendukung as dok inner join Tugas_akhir as ta on ta.tugas_akhir_id = dok.tugas_akhir_id inner join mahasiswa as mah on mah.mahasiswa_id = dok.mahasiswa_id";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetchAll();
        return $result;
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function add($tugas_akhir, $file): bool
    {
        try {
            $query = "INSERT INTO Dokumen_pendukung (tugas_akhir_id, tanda_terima_ta, tanda_terima_pk1, bebas_kompen, status_dokumen_pendukung) VALUES (" .
                ($tugas_akhir, "" .
                $file["tanda_terima_ta"]["name"] . ", " .
                $file["tanda_terima_pk1"]["name"] . ", " .
                $file["bebas_kompen"]["name"] . ", " . status::PENDING->value . ")";
            $this->koneksi->KoneksiDB()->query($query);
            return true;
        } catch (PDOException $e) {

```

- View: View/dokumen



- Route: Routes/route.php dan Routes/routePendukung.php

```

if ($page == 'dokumenpendukung') {
    $pendukung->route();
}

class routePendukung extends Route{
    2 references
    private $DokumenPendukung;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->DokumenPendukung = new PendukungController();
        $this->routes = [
            'manageDokumen' => 'index',
            'getAll' => 'getAll',
            'addDokumen' => 'add',
            'getOne' => 'getOne',
        ];
    }
}

```

4.6. Fitur Management Bebas Tanggungan

Fitur ini memungkinkan user bisa mengakses dan memodifikasi data Bebas Tanggungan yang mencakup tambah, edit, hapus, dan verifikasi dokumen Bebas Tanggungan.

Letak Source Code:

- Controller: controller/BebasTanggunganController.php
Berisi beberapa fungsi untuk menunjang proses management Bebas Tanggungan seperti fungsi addUser(), getAll(), dan hapus().

```
class BebasTanggunganController
{
    9 references
    private $bebasTanggungan;
    4 references
    private $tugasAkhir;
    3 references
    private $mahasiswa;
    2 references
    private $dokumenPendukung;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->bebasTanggungan = new BebasTanggungan();
        $this->tugasAkhir = new TugasAkhir();
        $this->dokumenPendukung = new DokumenPendukung();
        $this->mahasiswa = new Mahasiswa();
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): void
    {
        $data = $this->bebasTanggungan->getAll();
        if ($data) {
            header(header: 'Content-Type: application/json');
            echo json_encode(value: $data);
        } else {
            -
        }
    }
}
```

- Model: Model/BebasTanggungan.php
Berisi sebuah query sql yang menghubungkan antara sistem dan database untuk keperluan fitur management Dokumen Tugas Akhir.

```
class BebasTanggungan
{
    7 references
    private $koneksi;

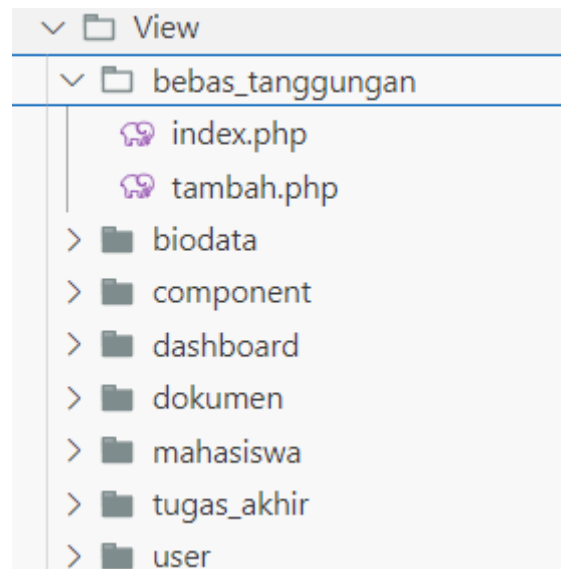
    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct()
    {
        $this->koneksi = new Koneksi();
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getAll(): array
    {
        $query = "SELECT * FROM Bebas_tanggungan as bt inner join Tugas_akhir as ta on ta.tugas_akhir_id = bt.tugas_akhir_id inner join Mahasiswa";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetchAll();
        return $result;
    }

    2 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function getOne($id): mixed
    {
        $query = "SELECT * FROM Bebas_tanggungan as bt inner join Tugas_akhir as ta on ta.tugas_akhir_id = bt.tugas_akhir_id inner join Mahasiswa";
        $data = $this->koneksi->KoneksiDB()->query($query);
        $result = $data->fetch();
        return $result;
    }

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function add($tugas_akhir, $no_surat): bool
    {
        try {
            $query = "INSERT INTO Bebas_tanggungan (tugas_akhir_id, no_surat, status_bebas_tanggungan) VALUES ('$tugas_akhir', '$no_surat', ''";
            $data = $this->koneksi->KoneksiDB()->query($query);
            return true;
        }
    }
}
```

- View: View/bebas_tanggungun



- Route: Routes/route.php dan Routes/routeBebasTanggungun.php

```

} elseif ($page == 'bebastanggungun') {
    $bebasTanggungun->route();
}

class routeBebasTanggungun extends Route {
    2 references
    private $bebasTanggungun;

    1 reference | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
    public function __construct() {
        $this->bebasTanggungun = new BebasTanggungunController();
        $this->routes = [
            'manageBebasTanggungun' => 'index',
            'getAll' => 'getAll',
            'verifikasi' => 'verifikasi',
            'add' => 'add',
            'getAllVerify' => 'getAllVerify',
            'downloadBebasTanggungun' => 'downloadBebasTanggungun',
        ];
    }
}

```

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Sistem backend yang dikembangkan berhasil mengotomatisasi proses administratif bebas tanggungan mahasiswa dengan pendekatan MVC yang terorganisir. Dengan otentikasi berbasis sesi server-side, sistem memastikan keamanan akses pengguna, sementara struktur folder yang rapi mempermudah kolaborasi dan pemeliharaan.

5.2. Saran

- Implementasi microservices untuk fitur-fitur utama agar sistem lebih scalable.
- Pemanfaatan container seperti Docker untuk konsistensi lingkungan development dan production.